Reinforcement Learning

Justus Piater

October 17, 2023

These notes loosely follow a textbook [Sutton and Barto 1998] and are partially inspired by lecture notes due to Andrew G. Barto. Most illustrations are taken from or inspired by these sources.

Contents

1 A Motivating Example **1**

2 Introduction to RL **2**

3 States, Actions, Rewards **7**

4 Markov Decision Processes **10**

5 Estimating Value Functions **16**

6 Temporal-Difference Learning **18**

7 Exercises **22**

8 References **24**

1 A Motivating Example

Tic-Tac-Toe [Slide 1]

Issues:

- What is the best *general* strategy of play?
- What is the best strategy against a *specific* opponent?
- At each of my turns, how do I decide on my move?
  I need a ***policy*** that maps ***states*** to ***actions***.
- I know the rules, but I do not necessarily have a teacher.
  I receive no instructions, only ***rewards***.
- The consequences of my move may not materialize until much later.
  This is the problem of ***delayed rewards***.

Solving Tic-Tac-Toe [Slide 2]

- Minimax search: *Maximize* my chances of winning given that my oppenent *minimizes* my chances of winning, by walking the game tree.

X	O	
X		X
O	O	

  This can be sped up using alpha-beta pruning.

  Note

  Tic-Tac-Toe can only be solved in this way because its game tree is manageably small.

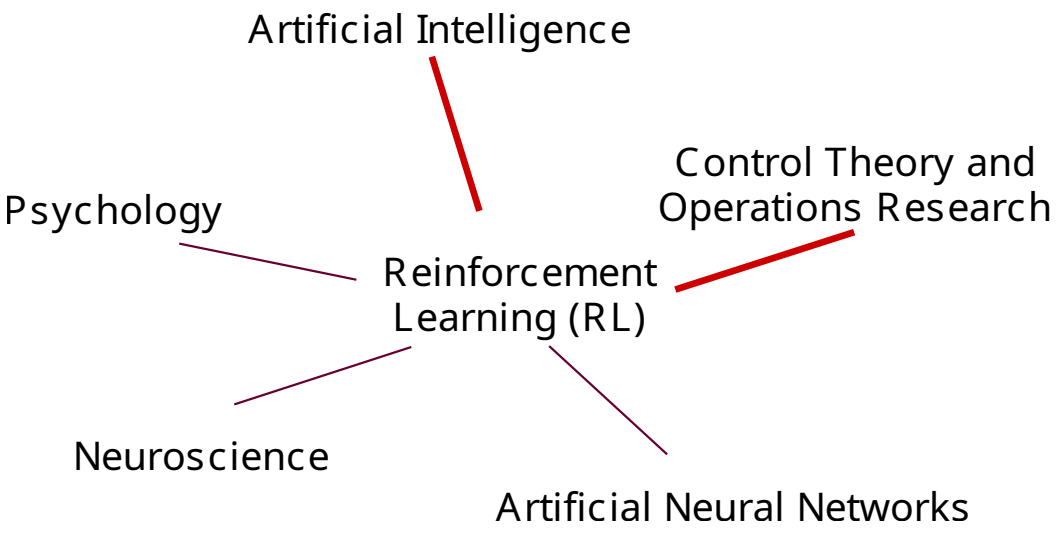
- ***Reinforcement Learning***: Learn a policy that maximizes ***returns*** by *trial and error*, and observing the resulting rewards.

2 Introduction to RL

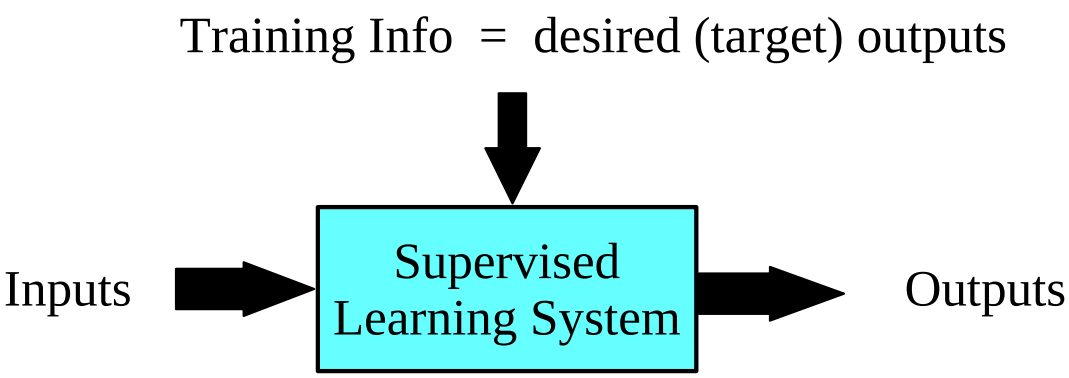
What is RL? [Slide 3]

- Learning by trial and error
- Goal-oriented learning
- Learning about, from, and while interacting with an external environment
- Learning what to do – how to map situations to actions – so as to maximize a numerical reward signal

Origins of RL [Slide 4]

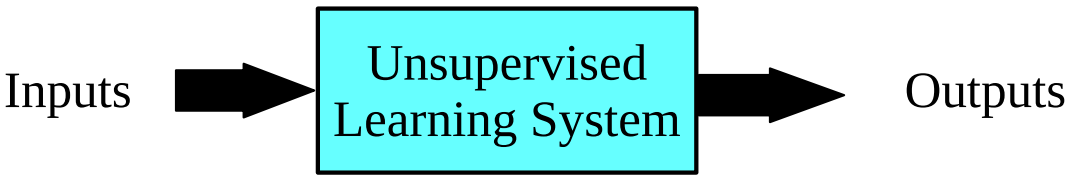


Supervised Learning [**Slide 5**]



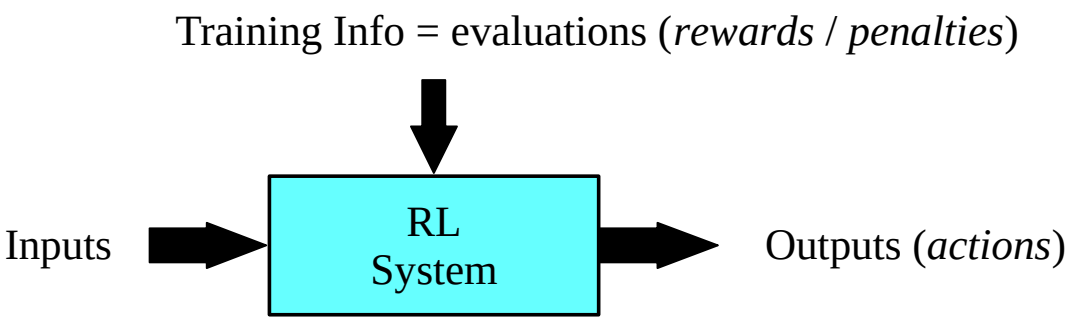
Objective: Minimize Error = ‖ target output − actual output ‖

Unsupervised Learning [**Slide 6**]



Objective: Maximize some objective function

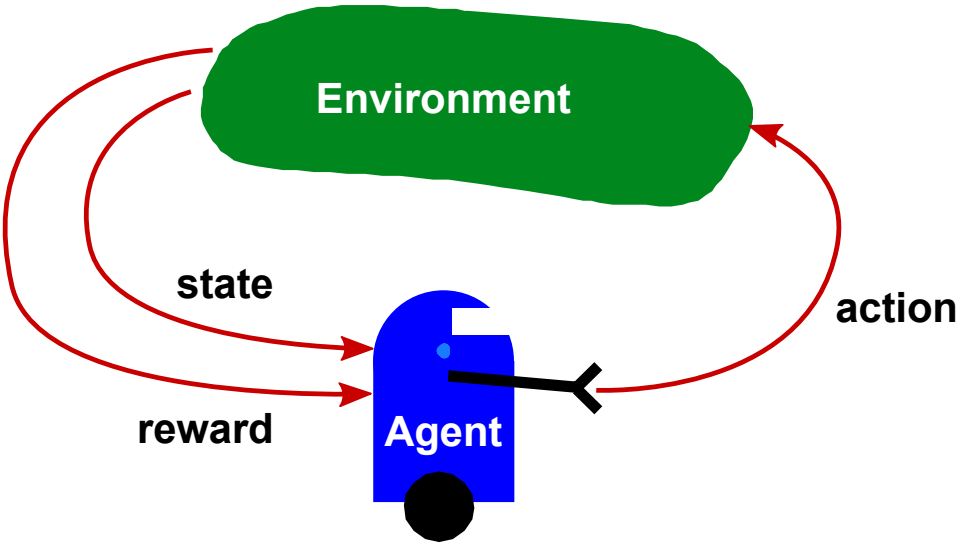
Reinforcement Learning [**Slide 7**]



Objective: Accumulate as much reward as possible

The RL Agent [Slide 8]

- Temporally situated
- Continual learning and planning
- Objective is to *affect* the environment
- Environment is stochastic



Key Features of RL [Slide 9]

- Learner is not told which actions to take

  Trial-and-Error search

- Possibility of *delayed reward*

  Sacrifice short-term gains for greater long-term gains

- The need to *explore* and *exploit*

- Considers the whole problem of a goal-directed agent interacting with an uncertain environment

Some Notable RL Applications [Slide 10]

- TD-Gammon [Tesauro 1995]

  World's best backgammon program

- Elevator Control [Crites and Barto 1995]

  high performance down-peak elevator controller

- Dynamic Channel Allocation [Singh and Bertsekas 1996, Nie and Haykin 1999]

  high performance assignment of radio channels to mobile telephone calls

Autonomous Helicopter Flight [Slide 11]

[Ng et al. 2004]

Quadruped Walking [Slide 12]

Before Learning

After Learning

[Kohl and Stone 2004]

Motor Skill Learning [Slide 13]

[Kober and Peters 2008]

The n-armed Bandit Problem [Slide 14]

Choose repeatedly from one of n actions; each choice is called a *play*.

After each play a_t, you obtain a reward r_t. The unknown distribution of r_t depends only on a_t, allowing you to estimate ***action values*** $Q^*(a_t) = E[r_t \mid a_t]$.

The Exploration/Exploitation Dilemma [Slide 15]

Your objective is to maximize the long-term cumulative reward.

To solve this problem, you must

- ***explore*** a variety of actions, and
- ***exploit*** the best of them.

Suppose you form action value estimates $Q_t(a) \approx Q^*(a)$.

If the greedy action is $a_t^* = \text{argmax}_a Q_t(a)$, then

- $a_t = a_t^*$ means exploitation,
- $a_t \neq a_t^*$ means exploration.

You can never stop exploring – but you should generally reduce exploring.

Some Action-Selection Strategies [Slide 16]

Greedy Action Selection:

$$a_t = a_t^* = \operatorname*{argmax}_a Q_t(a)$$

ϵ-Greedy Action Selection:

$$a_t = \begin{cases} a_t^* & \text{with probability } 1-\epsilon \\ \text{random} & \text{with probability } \epsilon \end{cases}$$

Softmax Action Selection (using a *Gibbs* or *Boltzmann* distribution):

$$\pi(a) = \frac{e^{Q_t(a)/\tau}}{\sum_{b=1}^{n} e^{Q_t(b)/\tau}}$$

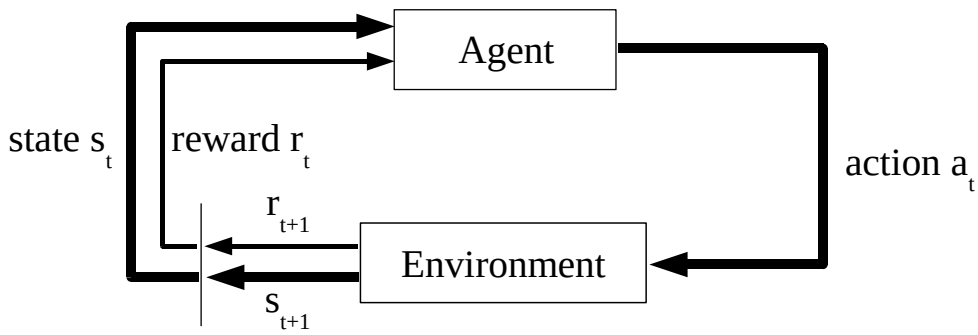
where $\tau > 0$ is the *temperature*

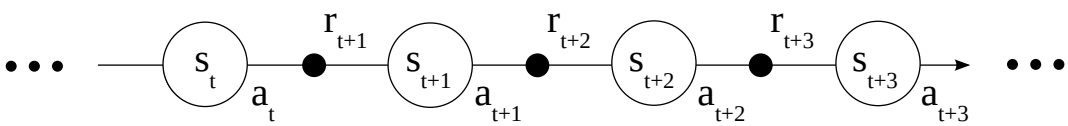
greedy $0 \leftarrow \tau \rightarrow \infty$ equiprobable

Optimistic initial values with greedy action selection

3 States, Actions, Rewards

The Perception-Action Cycle [Slide 17]





Policies [Slide 18]

Policy at step t: a mapping from states to action probabilities

$$\pi_t(s,a) = P(a_t = a \mid s_t = s) = \pi_t(a \mid s)$$

- RL methods specify how the agent changes its policy in response to experience.
- Roughly, the agent's goal is to receive as much reward as it can over the long run.

Examples [Slide 19]

- TD-Gammon

  States: features extracted from board configurations

  Actions: moves

  Rewards: $+1$ win, 0 draw, -1 loss

- Elevator Control

  States: button states; positions, directions and motion states of cars; passengers in cars and halls

  Actions: stop at, or go by, next floor

  Rewards: -1 per time step for each person waiting

- Recycling Robot

  States: sensory information

  Actions: search for a soda can; wait for somebody to bring it a can; empty bin; recharge

  Rewards: small positive for each can; large negative if battery runs low; zero otherwise

- Tic-Tac-Toe

  States: For example, number board positions according to the *magic square* in which all rows, columns and diagonals sum to 15:

2	9	4
7	5	3
6	1	8

  Keep each of the 9 indices in exactly one of three lists (of X, O and unoccupied positions).

  Actions: ...

  Rewards: ...

Goals and Rewards [Slide 20]

- The rewards define what the agent is going to learn.
- Is a scalar reward signal an adequate notion of a goal?

  Maybe not, but it is extremely flexible.
- A goal should specify *what* we want to achieve, not *how* we want to achieve it.
- A goal must be outside the agent's direct control – thus outside the agent.
- The agent must be able to measure success:
  - explicitly,
  - frequently over its life span.

Returns [Slide 21]

Suppose the sequence of rewards after step t is:

$r_{t+1}, r_{t+2}, r_{t+3}, \cdots$

We want to maximize the ***expected return*** $E\{R_t\}$ for each step t.

For ***episodic*** tasks,

$$R_t = r_{t+1} + r_{t+2} + \cdots + r_T = \sum_{k=0}^{T} r_{t+k+1}$$

where T is a final time step at which a terminal state is reached.

Returns for Continuing Tasks [Slide 22]

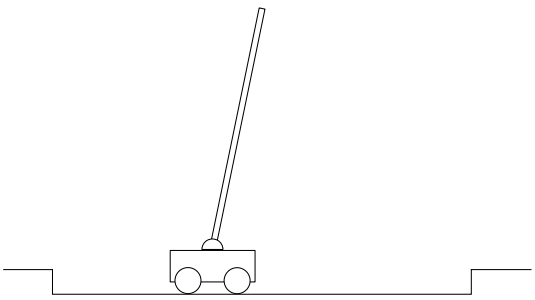
Discounted return:

$$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

where γ, $0 \leq \gamma < 1$, is the ***discount factor***.

shortsighted $0 \leftarrow \gamma \rightarrow 1$ farsighted

Example: Pole Balancing [Slide 23]



Avoid *failure*: the pole falling or the cart hitting an end of the track.

Episodic Task:

- reward $= +1$ for each step before failure
- return at each time = number of steps before failure

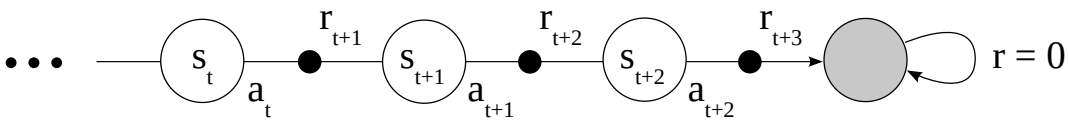
Continuing Task:

- reward $= -1$ upon failure; 0 otherwise
- return at each time related to $-\gamma^k$, for k steps before failure

In either case, the return is maximized by avoiding failure for as long as possible.

Unified Notation [Slide 24]

Think of each episode as ending in an ***absorbing state*** that always yields zero reward:



Thus, we can cover all cases by writing

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

where γ can be 1 only if a zero-reward absorbing state is always reached.

4 Markov Decision Processes

The Markov Property [Slide 25]

The ***state*** s_t at step t: whatever information is available to the agent about its environment at step t

immediate sensations, highly processed sensations, history of sensations, history of states, ...

Ideally, it should contain all *essential* information, i.e., it should possess the ***Markov Property***:

$$\begin{aligned} & P(s_{t+1} = s, r_{t+1} = r \mid s_t, a_t, r_t, s_{t-1}, r_{t-1}, \dots, r_1, s_0, a_0) \\ = \; & P(s_{t+1} = s, r_{t+1} = r \mid s_t, a_t) \end{aligned}$$

Markov Decision Processes [Slide 26]

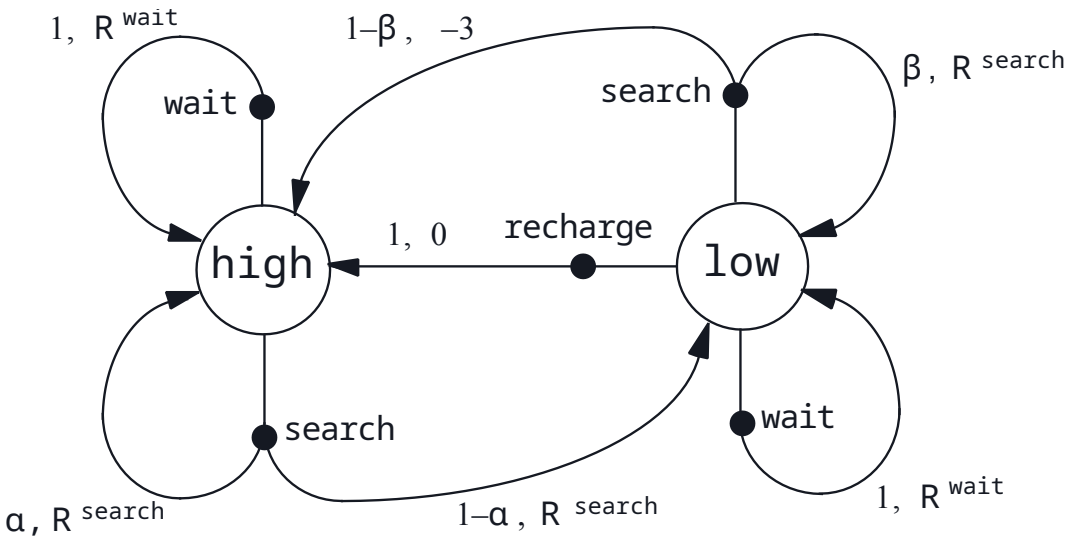
If an RL task has the Markov property, it is basically a ***Markov Decision Process (MDP)*** $(\mathcal{S}, \mathcal{A}, P_{\cdot\cdot}^{\cdot}, R_{\cdot\cdot}^{\cdot})$ with state set $\mathcal{S}$, action set $\mathcal{A}$, and transition and reward probabilities

$$\begin{aligned} P_{s,s'}^{a} &= P(s_{t+1} = s' \mid s_t = s, a_t = a) \\ R_{s,s'}^{a} &= E[r_{t+1} \mid s_t = s, a_t = a, s_{t+1} = s'] \end{aligned}$$

for all $s, s' \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

If $\mathcal{S}$ and $\mathcal{A}$ are finite, then it is called a ***finite MDP***.

Example: Recycling Robot Transition Graph [Slide 27]



Value Functions [Slide 28]

Value of a state: expected return starting from that state and following π

$V^{\pi}(s) = E_{\pi}[R_t \mid s_t = s]$

V^{π} is called the ***state-value function of policy*** π.

Estimating value functions: Maintain a V table, and maintain averages of observed returns. As the number of times a state is encountered approaches infinity, these averages will converge to the state values.

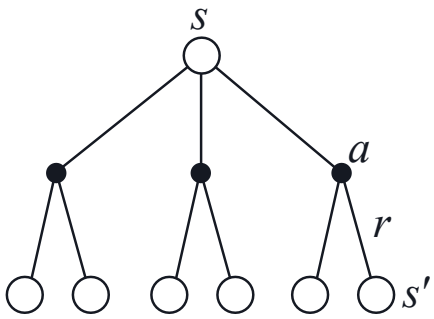
There is recursive structure that we can exploit:

$$\begin{aligned} R_t &= \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \\ &= r_{t+1} + \gamma \sum_{k=0}^{\infty} \gamma^k r_{t+k+2} \\ &= r_{t+1} + \gamma R_{t+1} \end{aligned}$$

The Bellman Equation for V^π [Slide 29]

$$
\begin{aligned}
V^\pi(s) &= E_\pi\left[R_t \mid s_t = s\right] \\
&= E_\pi\left[r_{t+1} + \gamma R_{t+1} \mid s_t = s\right] \\
&= E_\pi\left[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s\right]
\end{aligned}
$$

This is a system of linear equations (one for each state), which has a unique solution for V^π.



$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P^a_{s,s'} \left(R^a_{s,s'} + \gamma V^\pi(s')\right)$$

Backup diagram:

- shows all states, actions and rewards that participate in a "backup" (update of a value based on downstream values)
- time flows top to bottom
- states are not necessarily distinct (not a transition graph!)

Example: 2-State Random Walk [Slide 30]

- **States:**

  A and B

- **Actions:** stay or switch (taken with equal probabilities, with deterministic effect)

  Or equivalently: a single action with uniformly random effect

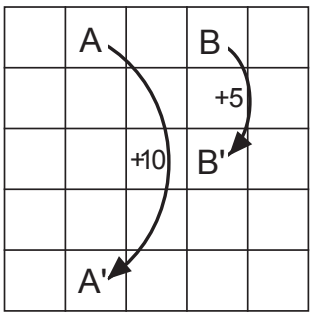
- **Rewards:**

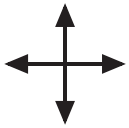
  $+2$ for a transition from A to B; -1 for a transition from B to A; 0 otherwise

- Discount factor $\gamma = 0.9$

$$
\begin{aligned}
V(A) &= \tfrac{1}{2}\left(2 + \tfrac{9}{10}V(B)\right) + \tfrac{1}{2}\left(0 + \tfrac{9}{10}V(A)\right) \\
V(B) &= \tfrac{1}{2}\left(-1 + \tfrac{9}{10}V(A)\right) + \tfrac{1}{2}\left(0 + \tfrac{9}{10}V(B)\right) \\
2V(A) &= 2 + \tfrac{9}{10}V(B) + \tfrac{9}{10}V(A) \\
V(A) &= \left(\tfrac{9}{10}V(B) + 2\right)\tfrac{10}{11} \\
V(B) &= \left(\tfrac{9}{10}V(A) - 1\right)\tfrac{10}{11} \\
V(A) &= \left(\tfrac{9}{10}\left(\tfrac{9}{10}V(A) - 1\right)\tfrac{10}{11} + 2\right)\tfrac{10}{11} = \left(\left(\tfrac{81}{100}V(A) - \tfrac{9}{10}\right)\tfrac{10}{11} + 2\right)\tfrac{10}{11} \\
&= \left(\tfrac{81}{110}V(A) + \tfrac{13}{11}\right)\tfrac{10}{11} = \tfrac{81}{121}V(A) + \tfrac{130}{121} = \tfrac{130}{121}\tfrac{121}{40} = 3\tfrac{1}{4} \\
V(B) &= \tfrac{9}{11}V(A) - \tfrac{10}{11} = \tfrac{9}{11}\tfrac{13}{4} - \tfrac{10}{11} = \tfrac{9\cdot 13 - 10\cdot 4}{44} = \tfrac{77}{44} = 1\tfrac{3}{4}
\end{aligned}
$$

Example: Gridworld [Slide 31]





Actions

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

- Rewards: -1 for actions that would take the agent off the grid; 0 otherwise, except for the two special cases shown; $\gamma = 0.9$
- Environment: deterministic as shown
- Policy: uniformly random

Notice the negative values near the edges, A's expected return less than its immediate reward, and B's expected return greater than its immediate reward.

Solved using the Bellman system of equations.

From State Values to Policies [Slide 32]

- Given a policy, we can compute the state values (*policy evaluation*).
- Given the state values, we can compute a better policy (*policy improvement*).
  Simply take actions that are greedy with respect to $E[r + \gamma V(s')]$.

Iterating these two steps yields the ***policy iteration*** RL algorithm.

This is our first RL algorithm!

Under the conditions considered here, it yields an optimal policy and an optimal value function.

Action Value Functions [Slide 33]

Value of taking an action in a state: expected return starting from that state, taking that action, and then following π

$$\begin{aligned}
Q^\pi(s,a) &= E_\pi[R_t \mid s_t = s, a_t = a] \\
&= E_\pi[r_{t+1} + \gamma R_{t+1} \mid s_t = s, a_t = a] \\
&= E_\pi[r_{t+1} + \gamma Q^\pi(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a] \\
&= E[r_{t+1} + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a]
\end{aligned}$$

Q^π is called the ***action-value function of policy*** π.

Optimal Value Functions [Slide 34]

For finite MDPs, policies can be *partially ordered*:

$\pi \geq \pi'$ iff $V^{\pi}(s) \geq V^{\pi'}(s)$ for all $s \in \mathcal{S}$

All *maximal* policies are called ***optimal policies*** π^*.

An optimal policy can be constructed by choosing, for each state, the best action from all possible policies. Thus, all optimal policies share the same optimal state-value and action-value functions:

$$\begin{aligned} V^*(s) &= \max_{\pi} V^{\pi}(s) && \text{for all } s \in \mathcal{S} \\ Q^*(s,a) &= \max_{\pi} Q^{\pi}(s,a) && \text{for all } s \in \mathcal{S},\ a \in \mathcal{A} \\ &= E[r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a] \end{aligned}$$

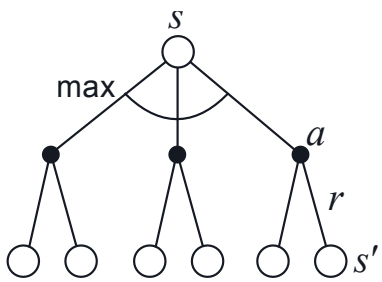
Note

$Q^*(s,a)$ is the expected return for taking action a in state s and thereafter following an optimal policy.

Bellman Optimality Equation for V^* [Slide 35]

The value of a state under an optimal policy must equal the expected return for the best action from that state:

$$\begin{aligned} & V^*(s) \\ = & \max_{a \in \mathcal{A}(s)} Q^{\pi^*}(s,a) \\ = & \max_{a \in \mathcal{A}(s)} E_{\pi^*}[r_{t+1} + \gamma V^*(s_{t+1}) \mid s_t = s, a_t = a] \\ = & \max_{a \in \mathcal{A}(s)} \sum_{s'} P^a_{s,s'} \left(R^a_{s,s'} + \gamma V^*(s') \right) \end{aligned}$$

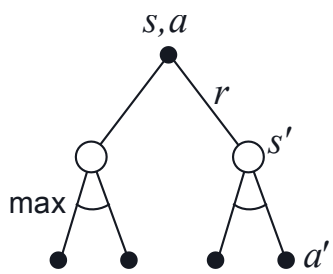


V^* is the unique solution of this system of nonlinear equations.

Bellman Optimality Equation for Q^* [Slide 36]

The value of an action in a given state must equal the expected return for following an optimal policy from that state:

$$\begin{aligned} & Q^*(s,a) \\ = & E\left[r_{t+1} + \gamma \max_{a' \in \mathcal{A}(s_{t+1})} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right] \\ = & \sum_{s'} P^a_{s,s'} \left(R^a_{s,s'} + \gamma \max_{a' \in \mathcal{A}(s')} Q^*(s', a') \right) \end{aligned}$$



Q^* is the unique solution of this system of nonlinear equations.

Optimal Value Functions Lead to Optimal Policies [Slide 37]

Any policy that is greedy with respect to an optimal value function is an optimal policy.

Given V^*, *one-step* look-ahead produces *long-term* optimal actions:

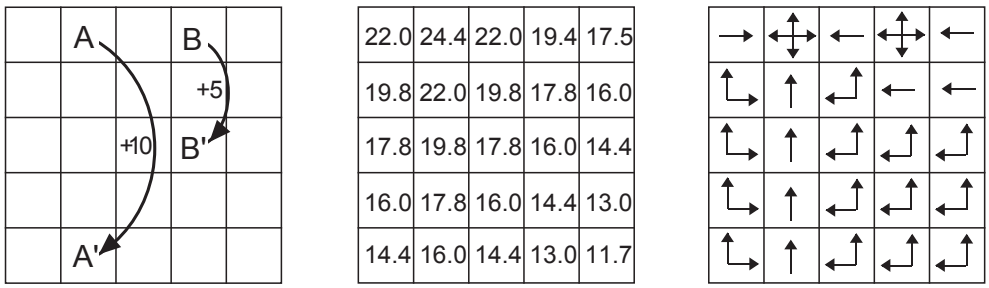
$$\pi^*(s) = \underset{a \in \mathcal{A}(s)}{\operatorname{argmax}} \sum_{s'} P^a_{s,s'} \left(R^a_{s,s'} + \gamma V^*(s') \right)$$

Note the notation: π^* is deterministic (unless the argmax is not unique)!

Given Q^*, *no* look-ahead is necessary at all:

$$\pi^*(s) = \underset{a \in \mathcal{A}(s)}{\operatorname{argmax}} Q^*(s, a)$$

Example: Gridworld [Slide 38]



a) gridworld b) V^* c) π^*

- Rewards: -1 for actions that would take the agent off the grid; 0 otherwise, except for the two special cases shown; $\gamma = 0.9$
- Environment: deterministic as shown
- Policy: *greedy*

5 Estimating Value Functions

Iterative (Dynamic Programming) Solutions [Slide 39]

- We have already seen the policy iteration algorithm.
- Turn the Bellman (optimality) equations into update rules, yielding an *iterative policy evaluation* and a ***value iteration*** algorithm, respectively.

$$
\begin{aligned}
V_{k+1}^{\pi}(s) &\leftarrow \sum_{a} \pi(a \mid s) \sum_{s'} P_{s,s'}^{a} \left(R_{s,s'}^{a} + \gamma V_{k}^{\pi}(s')\right) \\
V_{k+1}(s) &\leftarrow \max_{a} \sum_{s'} P_{s,s'}^{a} \left(R_{s,s'}^{a} + \gamma V_{k}(s')\right)
\end{aligned}
$$

Similarly for Q.

Value iteration is policy iteration with only one sweep of policy evaluation.

Note

One may proceed by full *sweeps*, or by *asynchronous* updates.

Practical Considerations [Slide 40]

- Finding an optimal policy by solving the Bellman Optimality Equation requires:
  - accurate knowledge of the environment dynamics,
    i.e., state transition probabilities
  - sufficient time and space,
  - the Markov property.
- How much time and space do we need?

  Both are polynomial in the number of states (with DP methods) – but this number is often huge (10^{20} for backgammon).
- We usually have to settle for approximations.

  (Many RL methods can be understood as approximately solving the Bellman Optimality Equation.)

Monte Carlo Policy Evaluation [Slide 41]

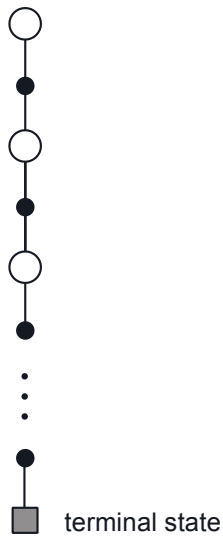
Goal: Learn $V^{\pi}(s) = E_{\pi}[R_t \mid s_t = s]$

Given: Episodes under π that contain s

Method: $V(s_t) \leftarrow V(s_t) + \alpha[R_t - V(s_t)]$

- *Weighted average* of one-shot and long-term estimates of V^{π}
- Learns from experience – *no model required* ($P_{\cdot\cdot}^{\cdot}$ and $R_{\cdot\cdot}^{\cdot}$)
- Independent state-value estimates
- No need to learn about *all* states

Only defined for *episodic* tasks.



A full, terminal epsiode as used here is often called a ***roll-out***.

Term coined by G. Tesauro in the context of TD-Gammon?

Without a model, this does not lead to a policy!

Monte Carlo Estimation of Action Values [Slide 42]

Goal: Learn $Q^{\pi}(s, a) = E_{\pi}[R_t \mid s_t = s, a_t = a]$

Given: Episodes under π that contain a taken in s

Method: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[R_t - Q(s_t, a_t)]$

Also converges asymptotically *if* every state-action pair is visited sufficiently often.

Exploration:

- ***Exploring starts***: Every state-action pair has a non-zero probability of being the starting pair.
- ***Soft*** policies: $\pi(a \mid s) > 0$ for all s and all a.

6 Temporal-Difference Learning

With Monte-Carlo methods, we do not need a model, but we need to wait for episodes to terminate.

With DP methods, we can do incremental updates based on other estimates (***bootstrap***), but we require a model.

Can we get the best of both worlds – bootstrap without a model? Yes We Can – with TD methods!

TD Prediction (Policy Evaluation) [Slide 43]

Recall: simple Monte Carlo

$V(s_t) \leftarrow V(s_t) + \alpha[R_t - V(s_t)]$

Target: the actual return after time t

The simplest TD method, TD(0):

$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$

Target: an estimate of the (expected) return

Tabular TD Prediction for estimating V^π [Slide 44]

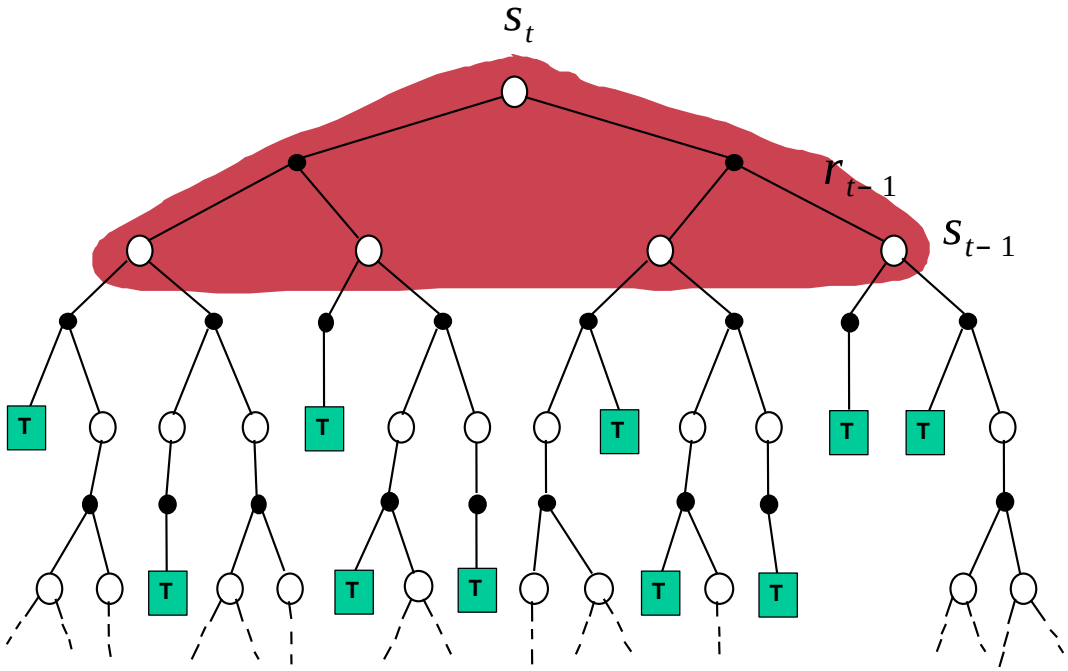
1. Initialize $V(s)$ arbitrarily
2. Repeat (for each episode):
   (a) Initialize s
   (b) Repeat (for each step):
      i. Choose a from s according to π
      ii. Take action a, observe r, s'
      iii. $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$
      iv. $s \leftarrow s'$

      until s is terminal

See the `gridworldTD0` demo.

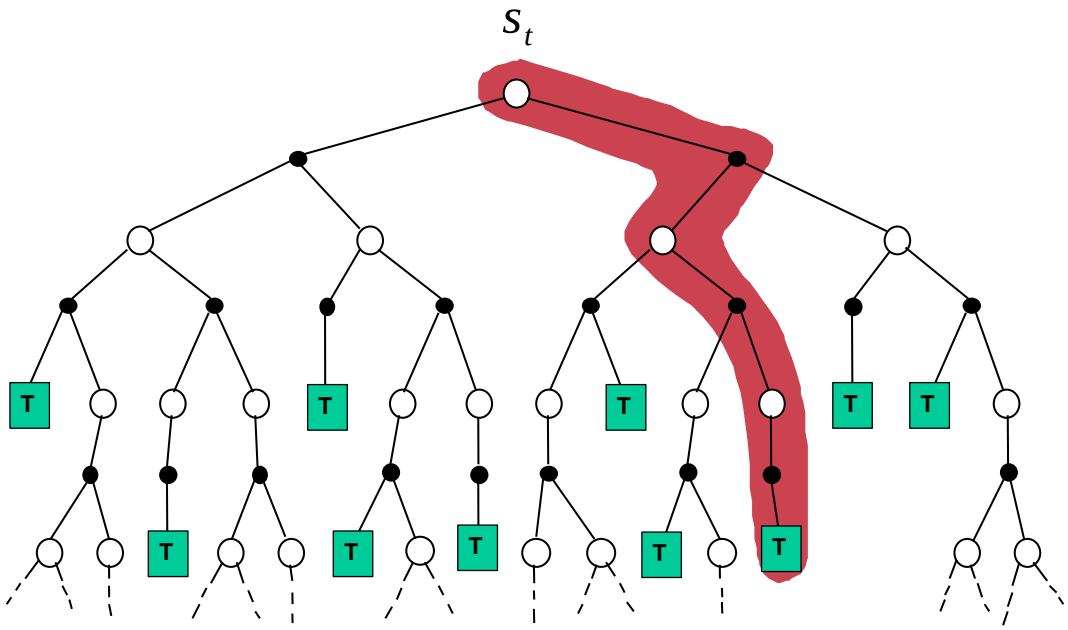
Dynamic Programming [Slide 45]

$V(s_t) \leftarrow E[r_{t+1} + \gamma V(s_{t+1})]$



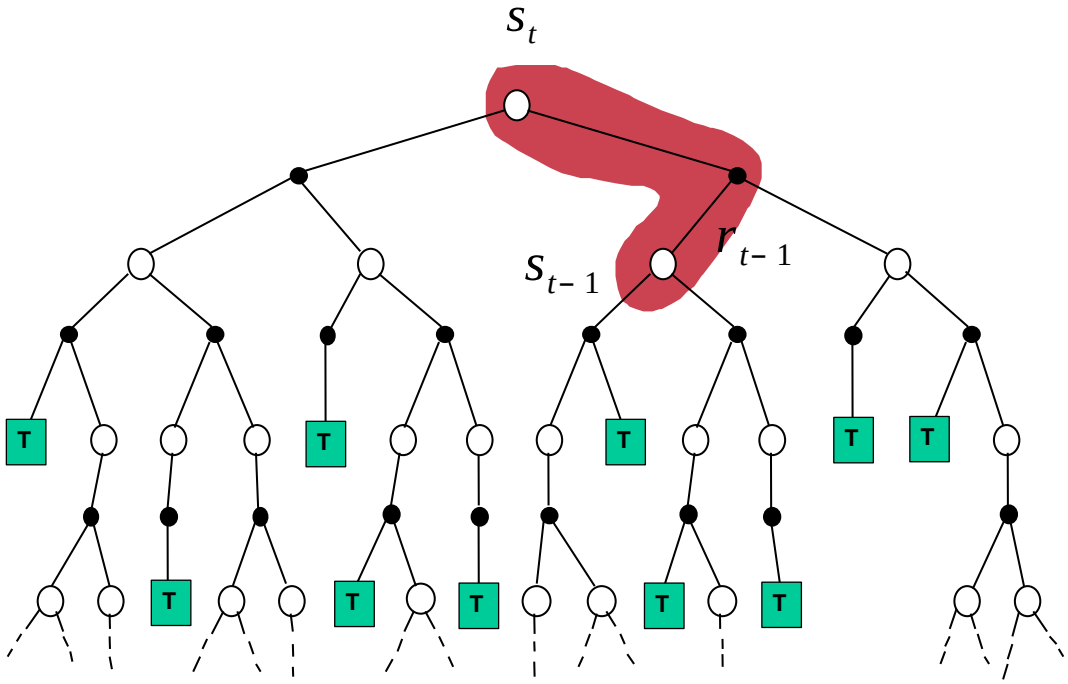
Simple Monte Carlo [Slide 46]

$V(s_t) \leftarrow V(s_t) + \alpha[R_t - V(s_t)]$



Simplest TD method [Slide 47]

$V(s_t) \leftarrow V(s_t) + \alpha[r_{t+1} + \gamma V(s_{t+1}) - V(s_t)]$



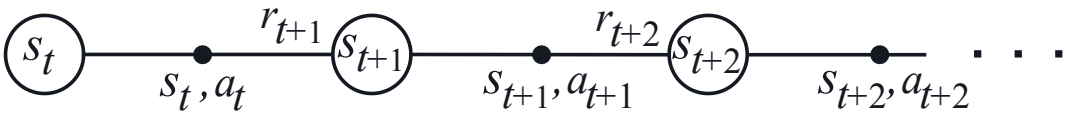
Advantages of TD Learning [Slide 48]

	DP	**MC**	**TD**
Bootstraps:	yes	no	yes
Samples:	no	yes	yes

- TD (and MC, but not DP) methods do not require a model of the environment, only experience.
- TD (and DP, but not MC) methods can be fully incremental:
  - You can learn *before* knowing the final outcome.
    (less memory, less peak computation)
  - You can learn *without* the final outcome.
    (from incomplete and non-episodic tasks)
- All three methods converge (under certain assumptions).

Learning an Action-Value Function [Slide 49]

Estimate Q^π for the current policy π.



After every transition from a nonterminal state s_t:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

This is the exact equivalent of TD(0) for action values.

Sarsa: On-Policy TD Control [Slide 50]

1. Initialize $Q(s, a)$ arbitrarily
2. Repeat (for each episode):
   (a) Initialize s
   (b) Choose a from s using policy derived from Q (e.g., ε-greedy)
   (c) Repeat (for each step):
      i. Take action a, observe r, s'
      ii. Choose a' from s' using policy derived from Q
      iii. $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
      iv. $s \leftarrow s'$, $a \leftarrow a'$

      until s is terminal

See the `gridworldSarsa` demo.

Q-Learning: Off-Policy TD Control [Slide 51]

Off-Policy RL: Learn the value function of an optimal policy while following a *different* policy.

1. Initialize $Q(s, a)$ arbitrarily
2. Repeat (for each episode):
   (a) Initialize s
   (b) Repeat (for each step):
      i. Choose a from s using (almost) any policy
      ii. Take action a, observe r, s'
      iii. $Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right]$

      Cf. Bellman Optimality Equation for Q^*.

      iv. $s \leftarrow s'$

      until s is terminal

See the `gridworldQ` demo.

7 Exercises

Tic-Tac-Toe [Slide 52]

Similar to Exercises 1.1–1.5 [Sutton and Barto 1998]

1. How would you train your Tic-Tac-Toe player: against itself, a minimax player, a random player, …?
2. What exploration strategy would you use? Do you need to explore at all?
3. Should your state representation exploit symmetries to increase efficiency?

An Episodic Task [Slide 53]

Exercise 3.5 [Sutton and Barto 1998]

4. You are designing a robot to run a maze as follows, using all-zero rewards, except for a unit reward on a successful escape from the maze, which ends an episode.

   After running the agent for a while, it shows no improvement in escaping from the maze.

   What is going wrong?

Markov Property [Slide 54]

5. Complete the definition of our pole-balancing example problem. Are your RL states Markovian?

   Example 3.5 [Sutton and Barto 1998]

6. Your RL agent perceives the world through cameras. Does it have access to the Markov state of its environment?

   Think about occlusions, objects behind the camera, etc.

   Exercise 3.6 [Sutton and Barto 1998]

Bellman Equation [Slide 55]

7. For the center state of our Gridworld example, verify the Bellman equation numerically:

   $$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P^{a}_{s,s'} \left(R^{a}_{s,s'} + \gamma V^{\pi}(s') \right)$$

   Exercise 3.9 [Sutton and Barto 1998]

8. What is the Bellman equation for action values, that is, for Q^{π}? It must give the action value $Q^{\pi}(s,a)$ in terms of action values, $Q^{\pi}(s',a')$, of possible successors to the state-action pair (s,a). Begin by drawing the backup diagram corresponding to this equation (cf. The Bellman Equation for V^{π}).

   Exercise 3.8 [Sutton and Barto 1998]

Bellman Optimality Equation [Slide 56]

9. Give the Bellman Optimality Equation for V^* for the recycling robot.

   Example 3.11 [Sutton and Barto 1998]

10. Give the Bellman Optimality Equation for Q^* for the recycling robot.

11. For the Gridworld example, use your knowledge of the optimal policy and the definition of the return to express the value of the best state symbolically, and then compute it to three decimal places.

    Exercise 3.17 [Sutton and Barto 1998]

Sampling-based RL Methods [Slide 57]

12. What is the backup diagram for Monte Carlo estimation of Q^π?

    Exercise 5.2 [Sutton and Barto 1998]

13. What is the backup diagram for Sarsa?

    Exercise 6.8 [Sutton and Barto 1998]

14. Why is Q-Learning considered an *off-policy* control method?

    Exercise 6.9 [Sutton and Barto 1998]

15. Consider the learning algorithm that is just like Q-Learning except that instead of the maximum over next state-action pairs it uses the expected value, taking into account how likely each action is under the current policy:

$$
\begin{aligned}
& Q(s_t, a_t) \\
\leftarrow\ & Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma E \left[Q(s_{t+1}, a_{t+1}) \mid s_t \right] - Q(s_t, a_t) \right] \\
\leftarrow\ & Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \sum_a \pi \left(s_t \mid a \right) Q(s_{t+1}, a) - Q(s_t, a_t) \right]
\end{aligned}
$$

    Is this new method an on-policy or off-policy method? What is the backup diagram for this algorithm? Given the same amount of experience, would you expect this method to work better or worse than Sarsa? What other considerations might impact the comparison of this method with Sarsa?

    Exercise 6.10 [Sutton and Barto 1998]

8 References

References [Slide 58]

Crites, Robert and Andrew Barto (1995). "Improving Elevator Performance Using Reinforcement Learning." In: *Neural Information Processing Systems*. Vol. 8. Cambridge, MA, USA: MIT Press, pp. 1017–1023.

Kober, Jens and Jan Peters (2008). "Policy Search for Motor Primitives in Robotics." In: *Neural Information Processing Systems 22*. URL: `http://www-clmc.usc.edu/publications/K/kober_NIPS2008.pdf`.

Kohl, Nate and Peter Stone (May 2004). "Policy Gradient Reinforcement Learning for Fast Quadrupedal Locomotion." In: *International Conference on Robotics and Automation*, pp. 2619–2624. URL: `http://www.cs.utexas.edu/users/pstone/Papers/bib2html-links/icra04.pdf`.

Ng, Andrew, Adam Coates, Mark Diel, Varun Ganapathi, Jamie Schulte, Ben Tse, Eric Berger, and Liang Liang (2004). "Inverted autonomous helicopter flight via reinforcement learning." In: *International Symposium on Experimental Robotics*. URL: `http://heli.stanford.edu/papers/iser04-invertedflight.pdf`.

Nie, Junhong and Simon Haykin (1999). "A dynamic channel assignment policy through Q-learning." In: *IEEE Transactions on Neural Networks* 10.6, pp. 1443–1455. URL: `http://web.mit.edu/juho/Public/thesis/thesis_docs/nie_haykin.pdf`.

Singh, Satinder and Dimitri Bertsekas (1996). "Reinforcement Learning for Dynamic Channel Allocation in Cellular Telephone Systems." In: *Neural Information Processing Systems*, pp. 974–980. URL: `http://books.nips.cc/papers/files/nips09/0974.pdf`.

Sutton, Richard and Andrew Barto (1998). *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press.

Tesauro, Gerald (Mar. 1995). "Temporal Difference Learning and TD-Gammon." In: *Communications of the ACM* 38.3, pp. 58–68.